

Troubleshooting Microservice Architectures



July 11, 2024
Peter Zaitsev,
Co-Founder at Coroot

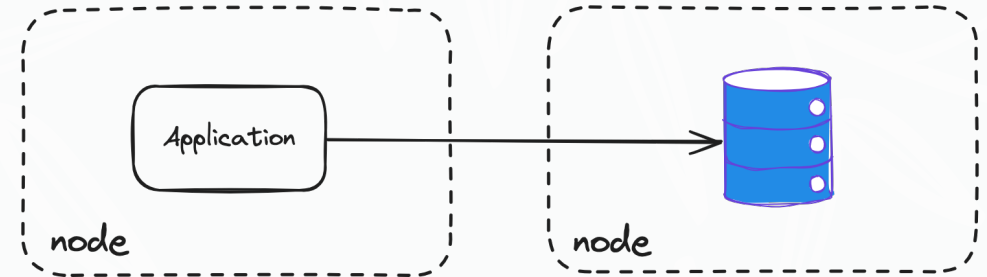


Nashville
DevOps
Days 2024

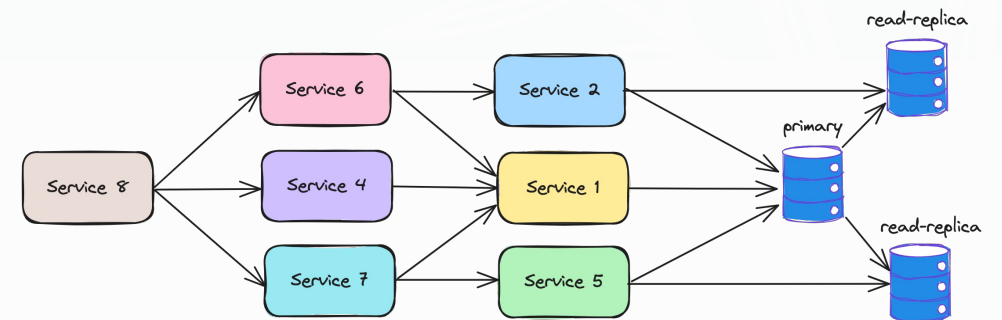
Troubleshooting with Observability

Evolution of the Applications

2000s



2020s



Monolith architecture (in theory)



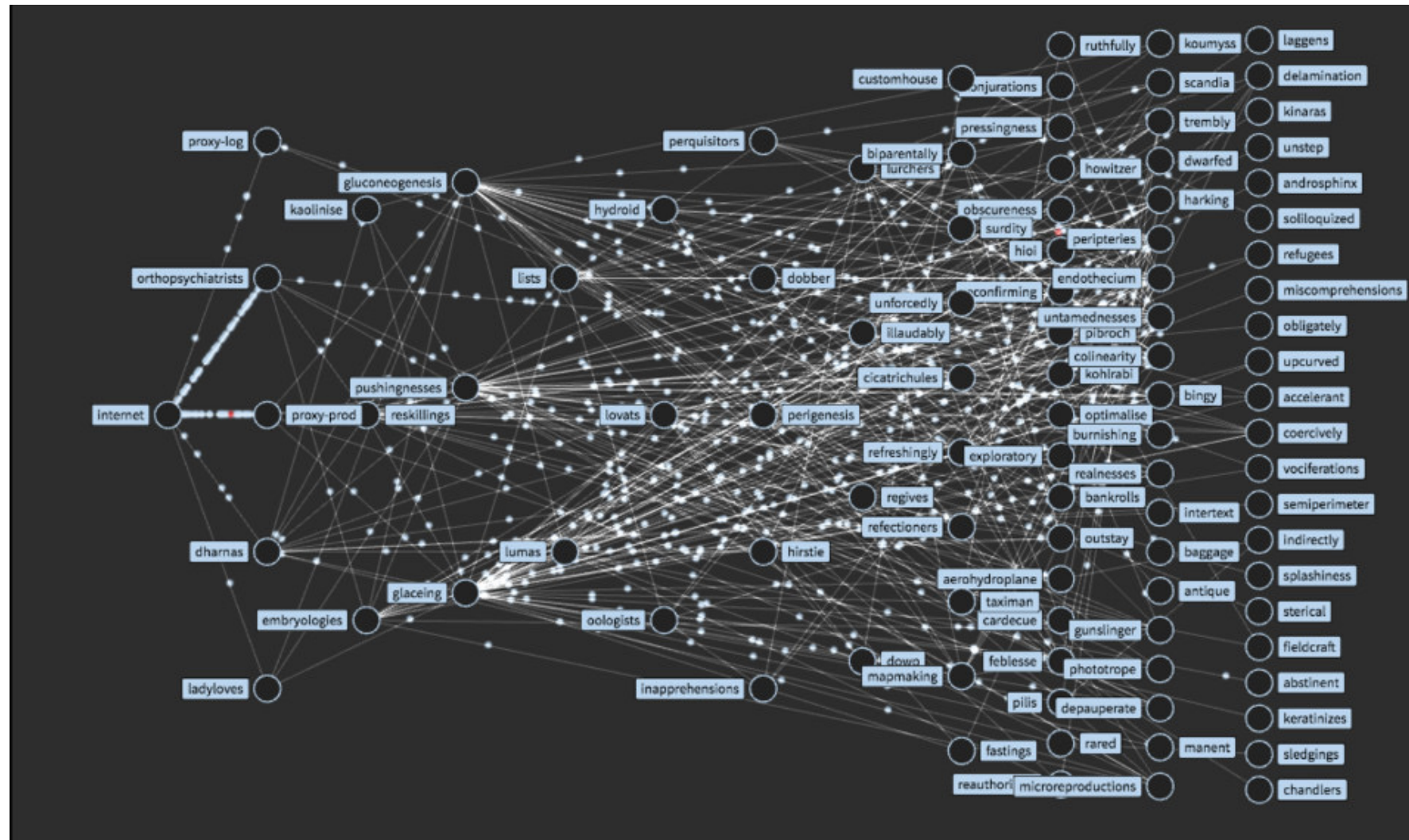
- Single, unified application
- Single tech stack
- Modularity through modules/packages
- In-app procedure calls
- Changes affect the entire application, slowing development
- Large, cross-functional teams
- Complexity increases with application size
- Less flexible, harder to adopt new technologies

Microservice Architecture (in theory)



- Collection of small, independent services
- Allows different technologies for different services
- Modularity through services
- Remote procedure calls through the network
- Faster development due to independent services
- Small, autonomous teams
- Manages complexity by breaking down into smaller services
- More flexible, easier to adopt new technologies

Microservice Architecture (real life)

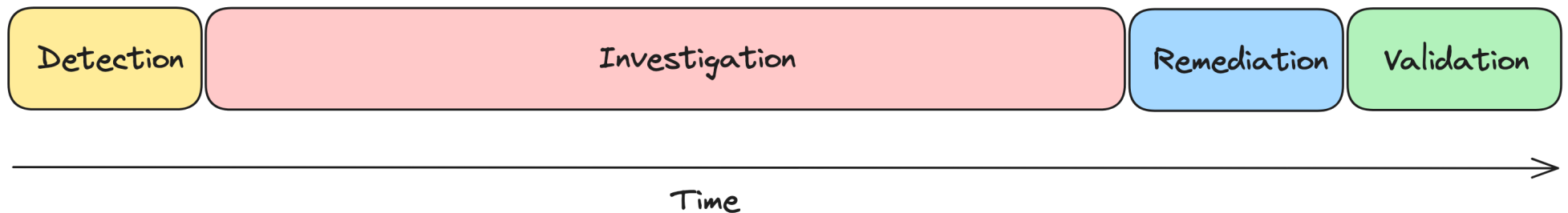


Microservice Architecture (real life)

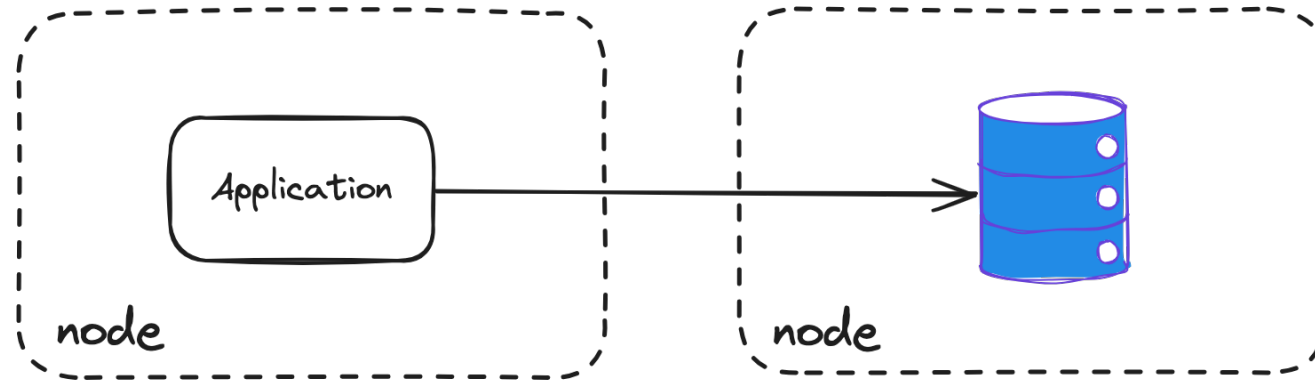
- Complexity shifted from development process to production environments
- A broad range of technologies in use:
 - *"We use Python, Java, NodeJS, Go, and we have some legacy services in PHP and Perl"*
 - *"Most likely, any database you can think of, we have it in production"*
- Shorter release cycle:
 - *"Our services are deployed 300 times a day"*
- Services communicate with each other through the network and often use DNS for service discovery
- Application instances run on shared nodes and compete for CPU, Memory, Disk I/O and Network Throughput

Troubleshooting steps

- **Detection:** detect that we have an issue
 - A service is not meeting its SLOs (Service Level Objectives) such as Availability and Latency
- **Investigation:** find the root cause (the most challenging and time-consuming part)
- **Remediation:** implement a fix
- **Validation:** validate that the issue is fixed

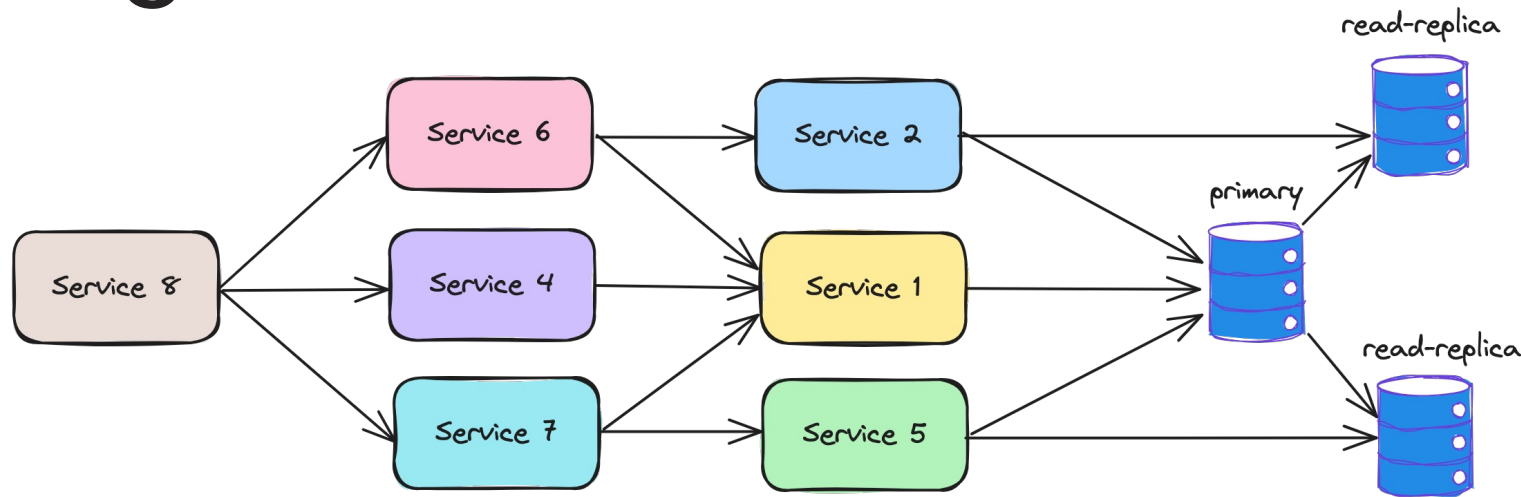


Investigation (monolith)



- `check(app)`
- `check(db)`
- If necessary, check the underlying nodes, network, etc.

Investigation (microservices)



- for service in services:
 - `check(service)`
- for db in databases:
 - `check(db)`

Since we can deal with hundreds or even thousands of services, the only way to manage this is to automate root cause analysis

Eat an elephant one bite at a time

- First, we should learn how to analyse any given service separately.
- Next, build a dependency graph.
- Then, traverse the graph and check services one by one, drilling down into their dependencies.
- Profit!





Organizational Issues

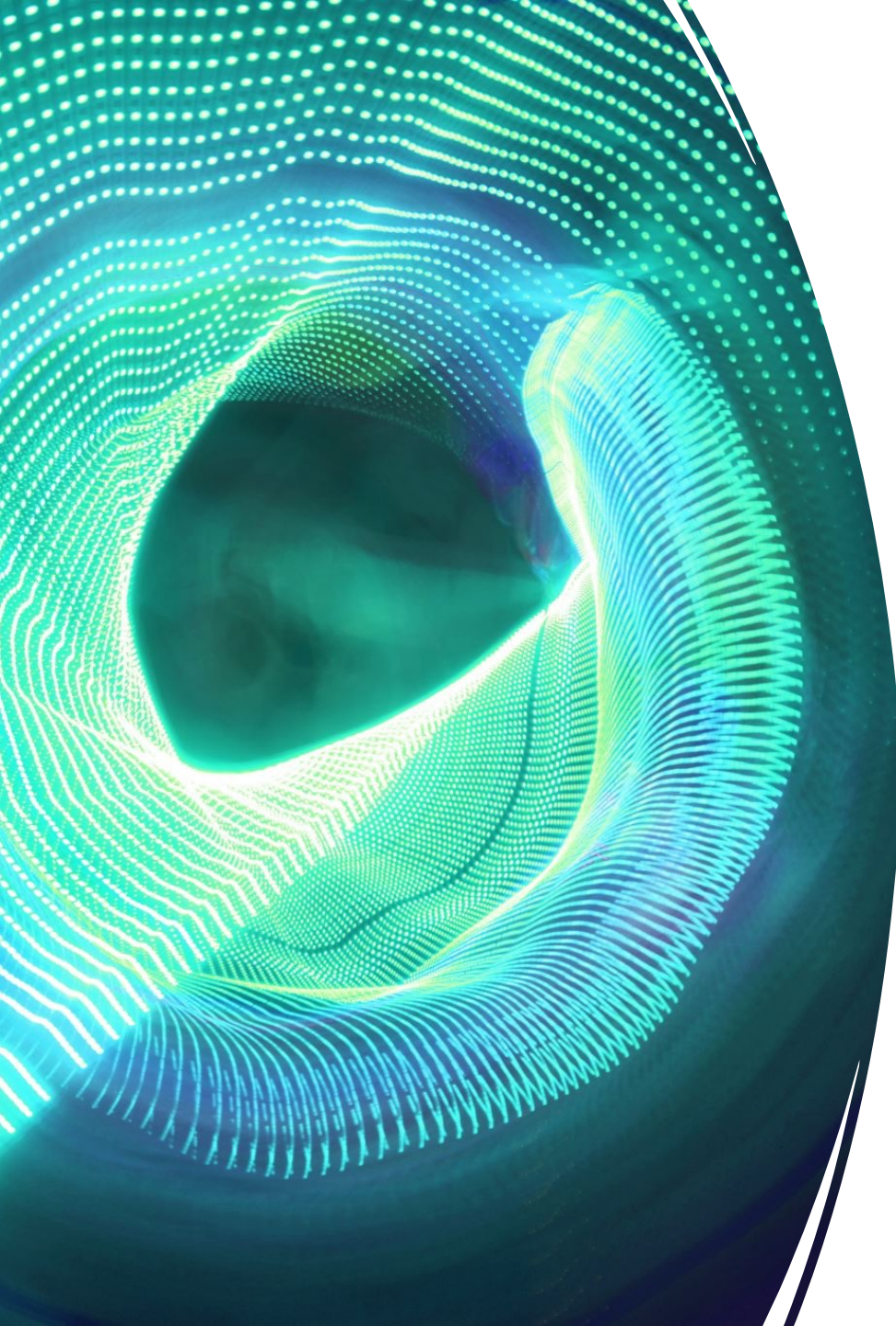
- Siloed Teams
- Siloed Tools
- Politics
- Vendor Responsibility
- Acceptance the Problem is with a component might be harder to achieve than resolution



What can happen with a service?

• ~~“There are countless options”~~ - to understand that this isn't true, let's write options down. Most likely, you won't be able to think of anything to add here!

- Instances: not running, crashed, restarted
- CPU: lack of CPU time because due to limits or node capacity
- Memory: OOM, slow allocation
- Disk: failure, I/O performance, can't write to disk (no space left)
- Runtime: garbage collection, thread pools, connection pools, locks
- Dependency services and databases: unavailability, high latency, errors
- Network to dependency services and databases: latency, packet loss
- DNS: it's always DNS
- Remaining 20% of issues that can be identified by errors in the log



The Cornerstones

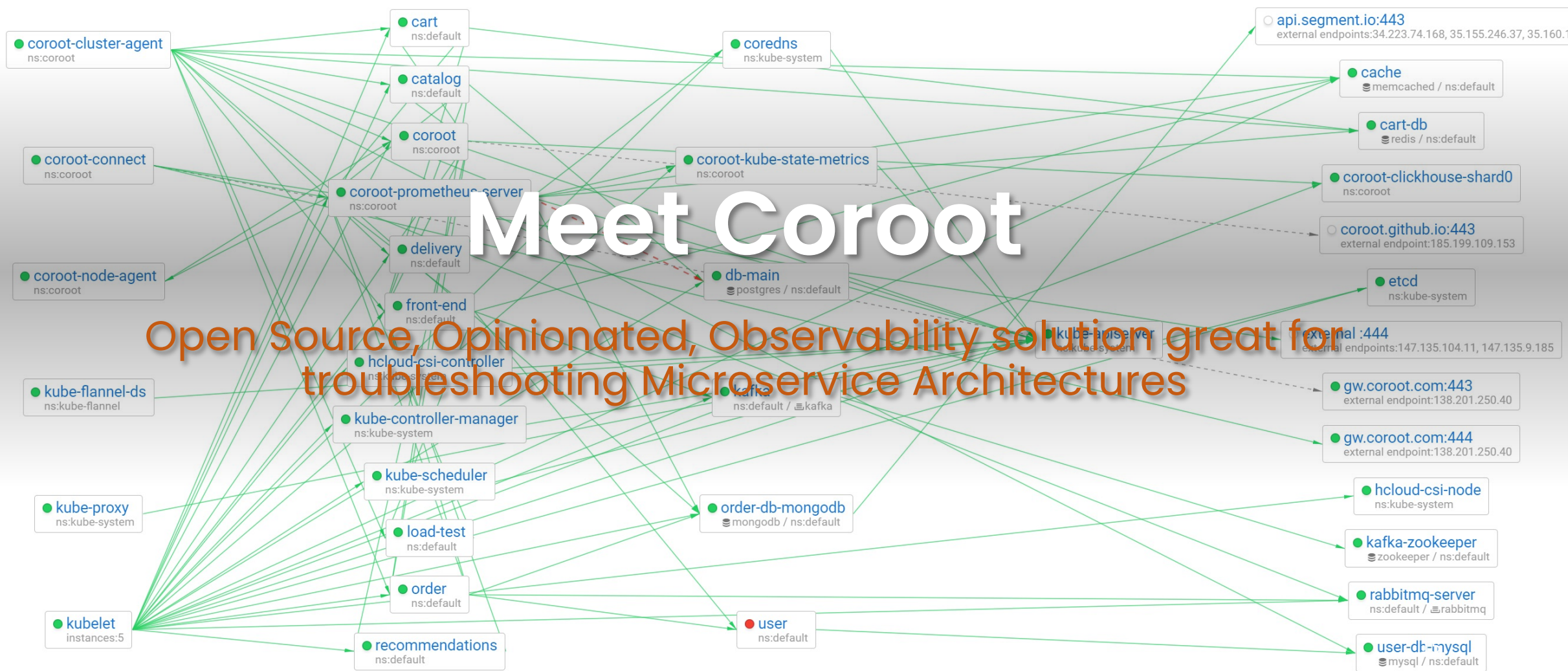
- Observability with no Blind spots
- Right Telemetry Data
- Dependencies between services
- Single tool for full picture observability
- Team cooperation

Overview

HEALTH SERVICE MAP TRACES NODES DEPLOYMENTS COSTS

search namespaces

☒ application ☒ control-plane ☒ monitoring ☐ other

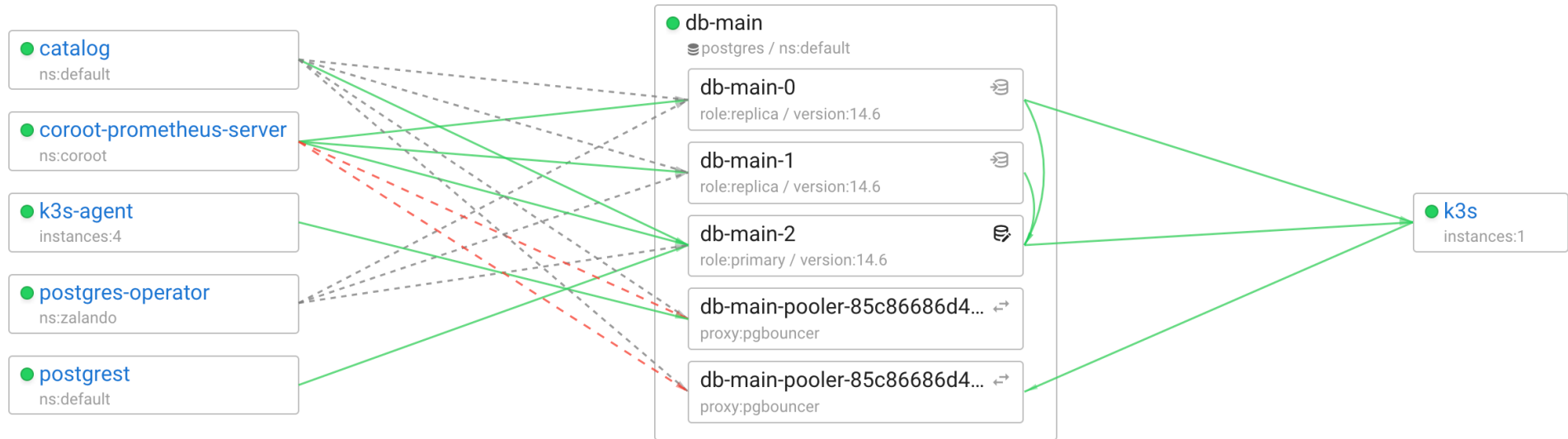




Coroot Highlights

- eBPF Based – Automagically instrument everything with zero code changes
- Simple, Actionable insights rather than dashboards overload
- Supports, Kubernetes, VM, Cloud, Bare Metal On Prem Deployments
- Integration with popular Cloud Providers for auto discovery and cost analyses
- Advanced Instrumentation support for most popular Open Source Databases
- Using Prometheus and Clickhouse for backend for scalable and efficient data store
- Open Source

Distributed systems are no longer black boxes



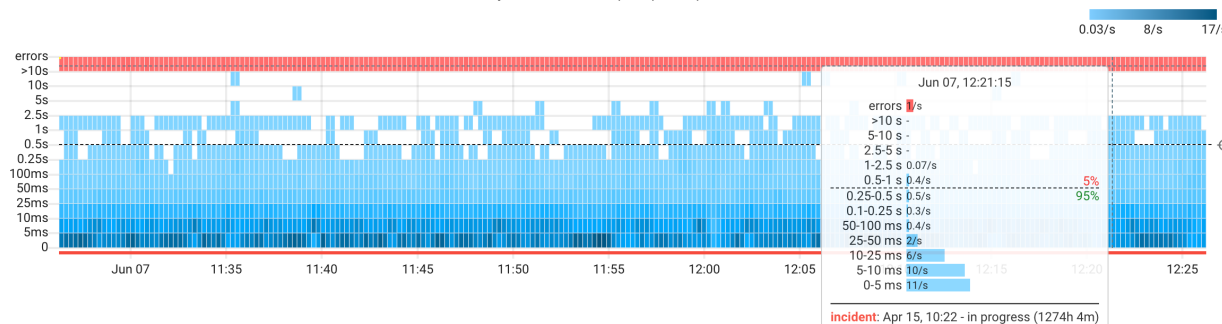
- L7: requests per second, Errors, Latency
- Network Round-trip-time (RTT)
- TCP: connections, failed connection attempts, retransmissions (can signify packet loss)

Inspecting an app: SLOs

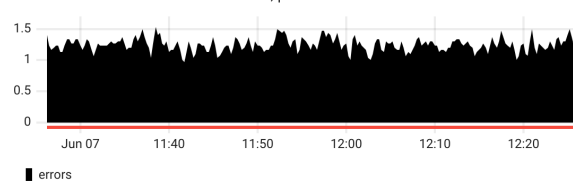
- Availability: error budget burn rate is 3.8x within 24 hours
Condition: the successful request percentage < 99%
- Latency: error budget burn rate is 0.7x within 1 hour
Condition: the percentage of requests served faster than 500ms < 99%

Client		Requests	Latency	Errors
coroot-cluster-agent http		0.2 /s	2041 ms	—
coroot-prometheus-server http		0.07 /s	20 ms	—
k3s-agent http		2 /s	0.5 ms	—
load-test http		30 /s	41 ms	1 /s

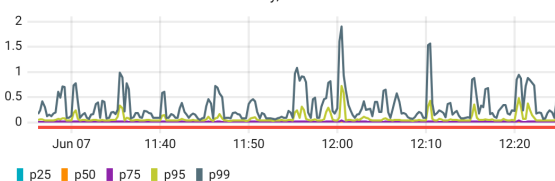
Latency & Errors heatmap, requests per second



Errors, per second



Latency, seconds

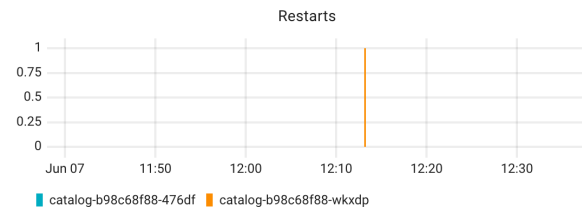
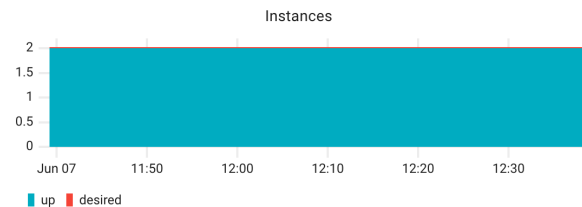


- **Availability:** 99% of request should not fail (**adjustable**)
- **Latency:** 99% of requests should be served in 500ms or less (**adjustable**)
- Alerts in the case of significant error budget burn rates

Inspecting an app: instances

- Instance availability: ok
Condition: the number of available instances < 75% of the desired
- Restarts: app containers have been restarted 1 time
Condition: the number of container restarts > 0

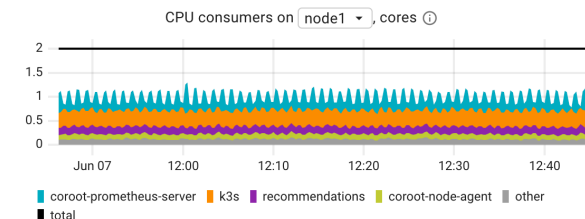
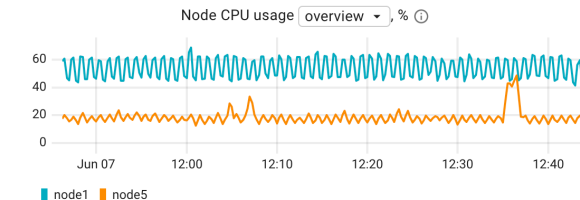
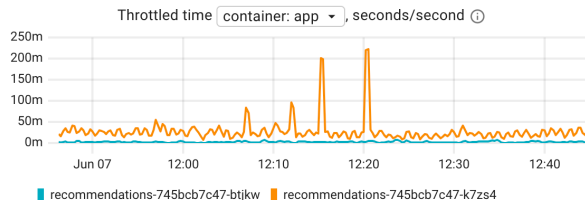
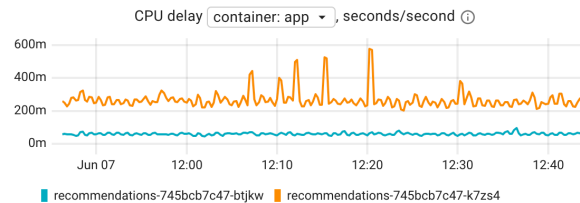
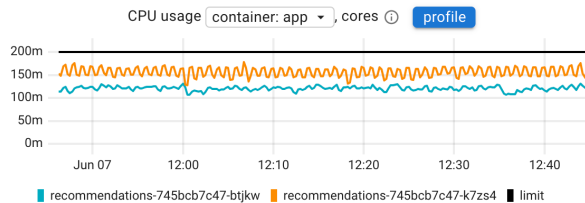
Instance	Status	Restarts	IP	Node
catalog-b98c68f88-476df	● up (running)	—	10.42.2.158	● node3
catalog-b98c68f88-wkxdp	● up (running)	1	10.42.3.91	● node5



- **Availability:** 75% of instances should be available (adjustable)
- **Restarts:** highlights instance restarts, because they can cause to request failures

Inspecting an app: CPU

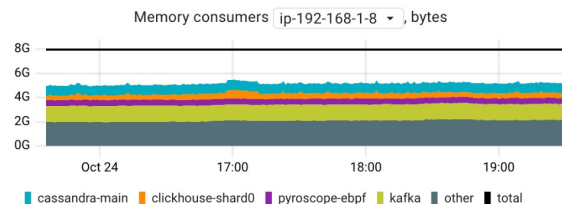
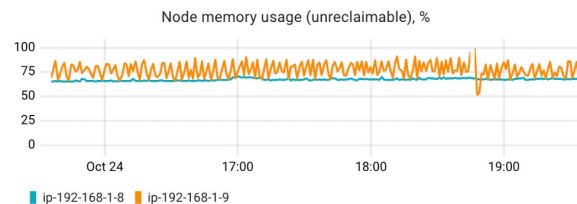
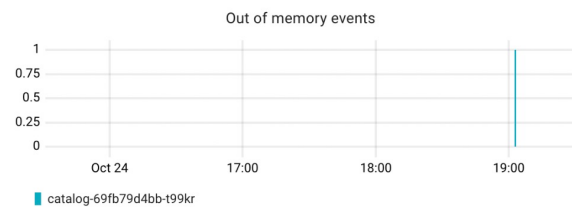
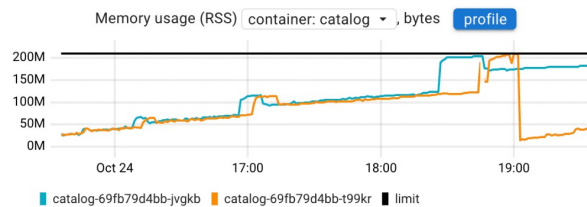
- Node CPU utilization: ok
Condition: the CPU usage of a node > 80%
- Container CPU utilization: high CPU utilization of 1 container
Condition: the CPU usage of a container > 80% of its CPU limit



- **Node CPU usage:** the relevant node CPU utilization >80% of CPU (**adjustable**)
- **Container CPU limit:** containers CPU usage >80% of their limits
- Easy to detect the noisy neighbor on any relevant node
- CPU delay: an added application latency caused by a lack of CPU time

Inspecting an app: memory

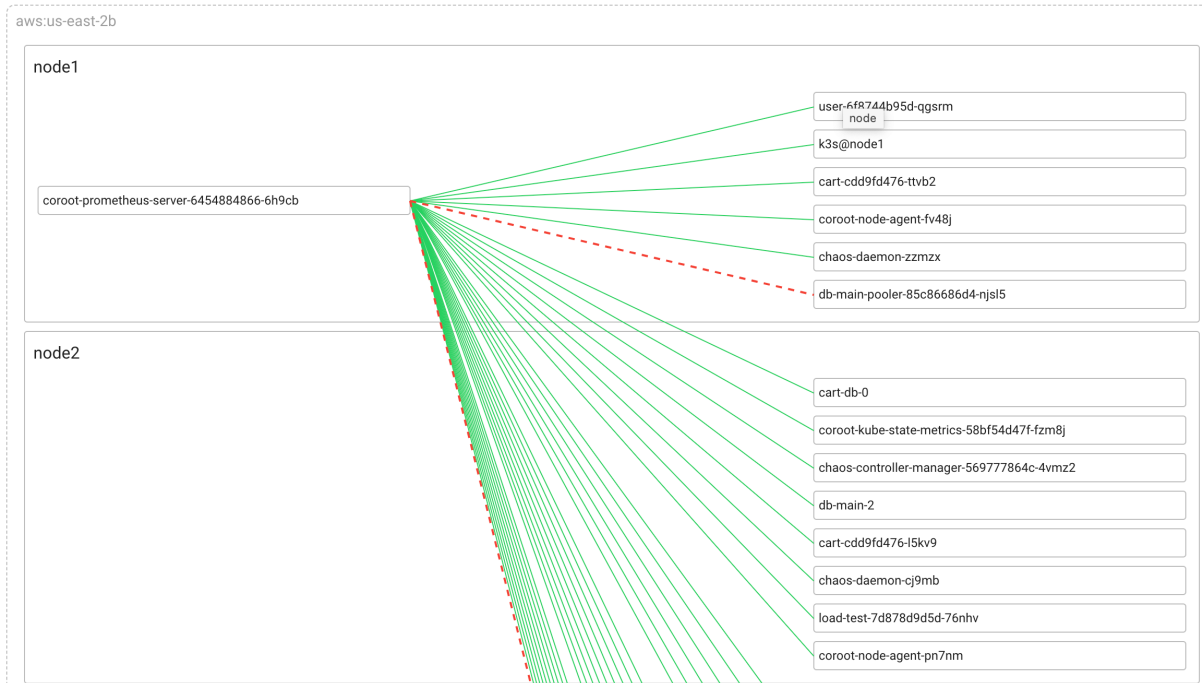
- Out of Memory: app containers have been restarted 1 time by the OOM killer
Condition: the number of container terminations due to Out of Memory > 0
- Memory leak: memory usage is growing by 22 % per hour
Condition: memory usage is growing by > 10 % per hour



- **Out of memory:** an instance was terminated by the OOM killer
- **Memory leak:** the usage is growing >10% per hour
- Easy to detect which app consumed most memory on any relevant node

Inspecting an app: network

- Network round-trip time (RTT): ok
Condition: the RTT to an upstream service > 10ms
- TCP connections: failed to connect to 1 upstream service
Condition: the number of upstream services to which the app failed to connect > 0
- Network connectivity: ok
Condition: the number of unavailable upstream services > 0

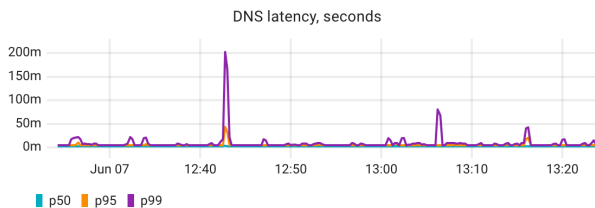
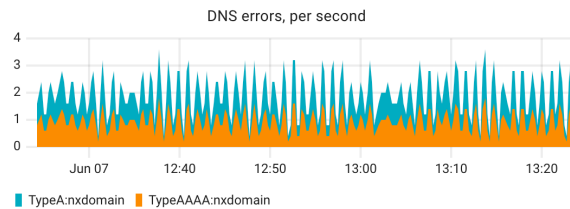
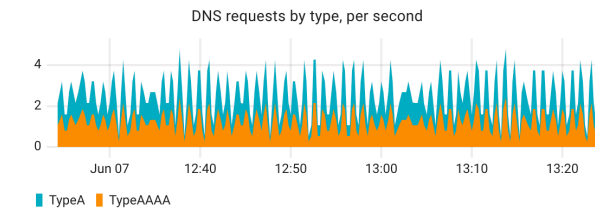


- **Network latency:** round-trip time to any upstream service should be < 10ms (**adjustable**)
- **Failed TCP connections:** tracks all connection from the app to its upstream services
- **Network connectivity:** uses ICMP probes to check connectivity between the app and its upstream services

Inspecting an app: DNS

- DNS latency: ok
Condition: the 95th percentile of DNS response times > 100ms
- DNS server errors: ok
Condition: the number of server DNS errors (excluding NXDOMAIN) > 0
- DNS NXDOMAIN errors: ok
Condition: the number of the NXDOMAIN DNS errors (for previously valid requests) > 0

Domain	Requests	No results (IPv4: A)	No results (IPv6: AAAA)	No results (other)	ServFail
○ api.segment.io.svc.cluster.local	2.3 k	1.1 k	1.1 k	—	—
○ api.segment.io.default.svc.cluster.local	2.3 k	1.1 k	1.1 k	—	—
○ api.segment.io.cluster.local	2.3 k	1.1 k	1.1 k	—	—
● api.segment.io	2.3 k	—	—	—	—
● order-db-arbiter-0.order-db-arbiter-headless.default.svc.cluster.local	2	—	—	—	—



- **DNS latency:** 95th percentile of DNS responses > 100ms (adjustable)
- **DNS server errors**
- **DNS NXDOMAIN errors:** highlights situations when there are no results in the response

Inspecting an app: logs

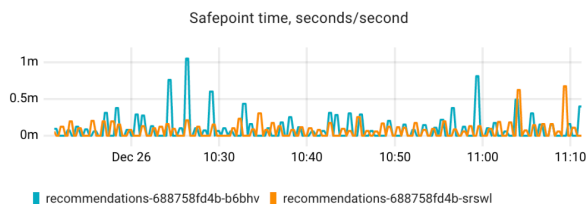
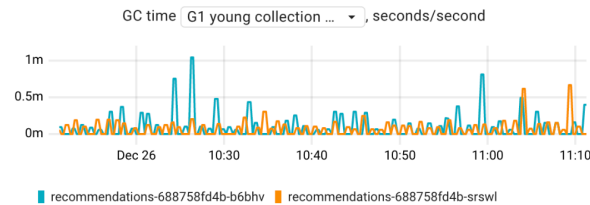
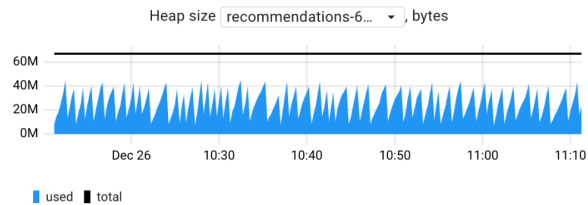


- **Errors:** checks whether errors in the app logs or not
- **Repeated patterns:** allow easily understand the unique errors in the log

Inspecting an app: JVM

- JVM availability: ok
Condition: the number of unavailable JVM instances > 0
- JVM safepoints: ok
Condition: the time application have been stopped for safepoint operations > 50ms

Instance	Status
recommendations-688758fd4b-b6bhv java: 18.0.2.1	● up
recommendations-688758fd4b-srswl java: 18.0.2.1	● up



- **JVM Safepoints:** stop-the-world pauses took >50ms per second (**adjustable**)

Inspecting an app: deployments (k8s)

● Deployment status: ok
Condition: a rollout is in progress > 3m

Deployment	Active	Summary
● d6986cb8d: catalog:0.26 17 hours ago	17 hours	🔥 CPU usage: -7% (compared to the previous deployment) ⓘ 💔 Memory: a memory leak detected (177MB per hour) ⓘ 💔 Logs: there are errors in the logs ⓘ
○ 7f96f8d465: catalog:0.25.2 18 hours ago	28 minutes	Not enough data due to the lifetime < 30 minutes
● 7767758c4b: catalog:0.25.1 19 hours ago	1 hour	🔥 Availability: 100% (objective: 99%) ⓘ 🔥 CPU usage: -17% (compared to the previous deployment) ⓘ
● 55846954f5: catalog:0.25 23 hours ago	3 hours	💔 Availability: 94.89% (objective: 99%) ⓘ 💔 CPU usage: +12% (compared to the previous deployment) ⓘ
○ 65dcb484cc: catalog:0.23 23 hours ago	12 minutes	Not enough data due to the lifetime < 30 minutes
● 7767758c4b: catalog:0.24 1 day ago	19 hours	💔 CPU usage: +5% (compared to the previous deployment) ⓘ
● 79b9947f7f 1 day ago	36 minutes	The deployment has been cancelled
● 65dcb484cc: catalog:0.23 2 days ago	23 hours	No notable changes

- **Rollouts:** detects if a rollout is stuck
- **Version comparison:** compares the new version of the app with the previous: CPU, Memory, Errors, SLIs

Dealing with thousands of services

HEALTH

SERVICE MAP

TRACES

NODES

DEPLOYMENTS

COSTS

🔍

search

namespaces

▼

☒ application

☐ control-plane

☐ monitoring

☐ other

+

1

SLO violation

0

Warning

3

Errors in logs

0

Integration required

13

OK

Application	Type	Errors	Latency	Upstreams	Instances	Restarts	CPU	Mem	I/O	Disk	Net	DNS	Logs
user	 python	–	991ms	1/1	3/3	–	–	–	–	–	0.7ms	–	–
front-end	 golang	–	817ms	6/7	1/1	–	–	–	–	–	0.6ms	–	2 unique errors
order-db-mongodb	 mongodb	–	79ms	–	3/3	–	–	–	<1%	31%	1ms	–	1 unique error
recommendations	 java	–	239ms	–	2/2	–	–	–	–	–	0.7ms	–	4 unique errors
cache	 memcached	–	7ms	–	1/1	–	–	–	–	–	–	–	–
cart	 golang	–	10ms	1/1	2/2	–	–	–	–	–	0.7ms	–	–
cart-db	 redis	–	7ms	–	1/1	–	–	–	–	<1%	–	–	–
catalog	 golang	–	61ms	1/1	2/2	–	–	–	–	–	0.5ms	–	–
catalog-db-init		–	–	–	–	–	–	–	–	–	–	–	–
db-main	 postgres	–	44ms	1/1	5/5	–	–	–	1%	6%	0.9ms	–	–
delivery	 golang	–	5ms	1/1	1/1	–	–	–	–	–	1ms	–	–
kafka	 kafka	–	41ms	–	3/3	–	–	–	<1%	10%	0.8ms	–	–
kafka-zookeeper	 zookeeper	–	–	–	3/3	–	–	–	–	<1%	0.7ms	–	–
load-test	 golang	–	–	1/1	1/1	–	–	–	–	–	<0.1ms	–	–
order	 golang	–	89ms	3/4	2/2	–	–	–	–	–	0.8ms	–	–
rabbitmq-server	 rabbitmq	–	–	–	1/1	–	–	–	–	–	<0.1ms	–	–
user-db-mysql	 mysql	–	43ms	–	2/2	–	–	–	3%	4%	0.7ms	–	–

Rows per page:

50

1-17 of 17

<

>

Alerts

- Only SLOs (Service Level Objectives) violations are sent as alerts
- Multi-window error budgeted burn rate

Severity	Long window	Short window	Burn rate threshold	Monthly error budget consumed
critical	1 hour	5 minutes	14.4	2% ≤ 50 hours to exhaustion
critical	6 hours	30 minutes	6	5% ≤ 5 days to exhaustion
warning	24 hours	2 hours	3	10% ≤ 10 days to exhaustion

- Every alert contains all detected issues related to the affected app:

 **Coroot** APP 2:28 PM
catalog is not meeting its SLOs

SLO:

- Latency: error budget burn rate is 97.2x within 1 hour

Network:

- Network round-trip time (RTT): high network latency to 1 upstream service

Logs:

- Errors: 2,376 errors occurred

[See less](#)

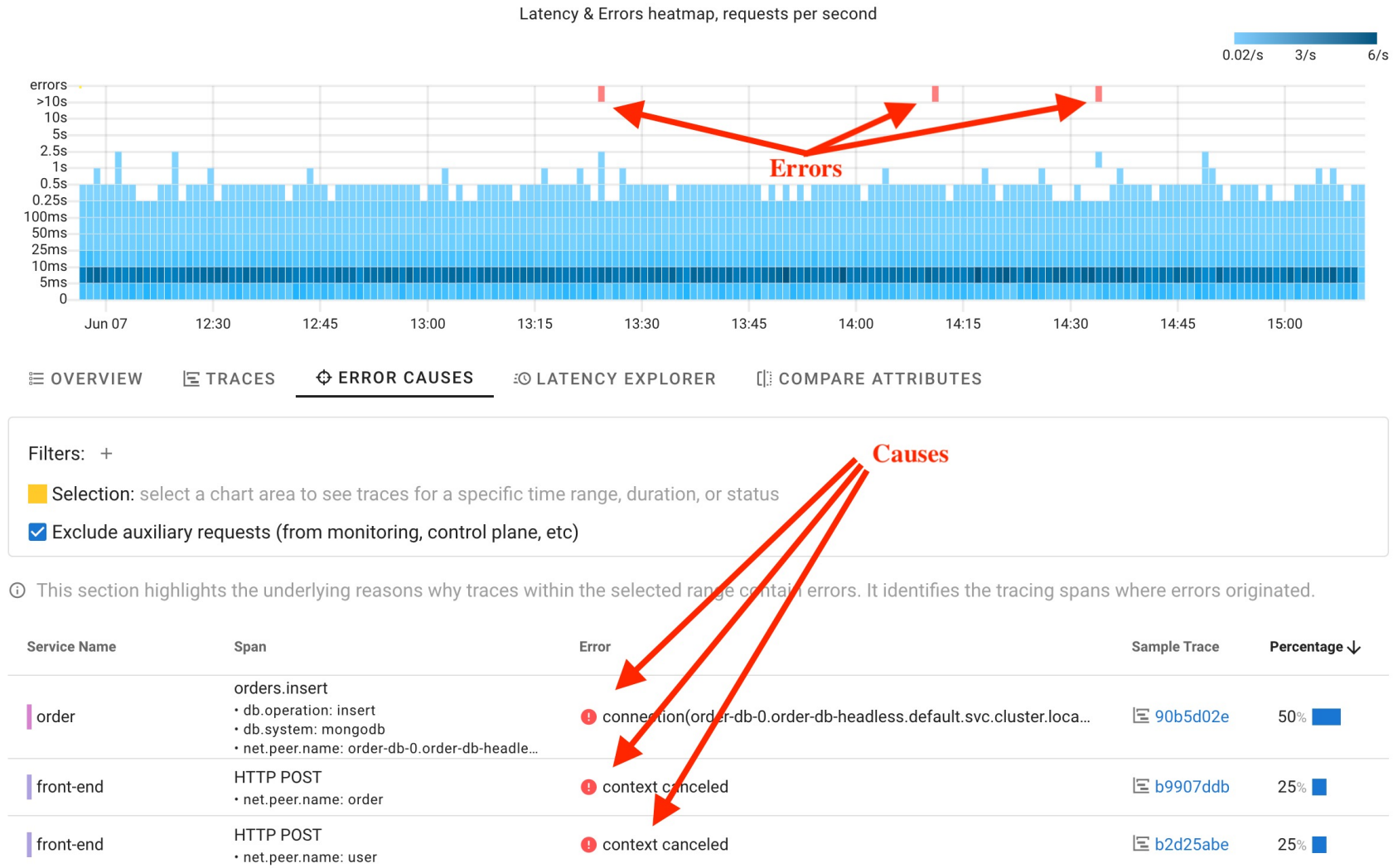
Distributed tracing (OpenTelemetry)

- eBPF-based instrumentation is a good starting point because it allows to see the big picture
- eBPF-based traces are not actually traces, they are disconnected spans because we don't have a TRACE_ID.
- Coroot can also be used as an OpenTelemetry backend

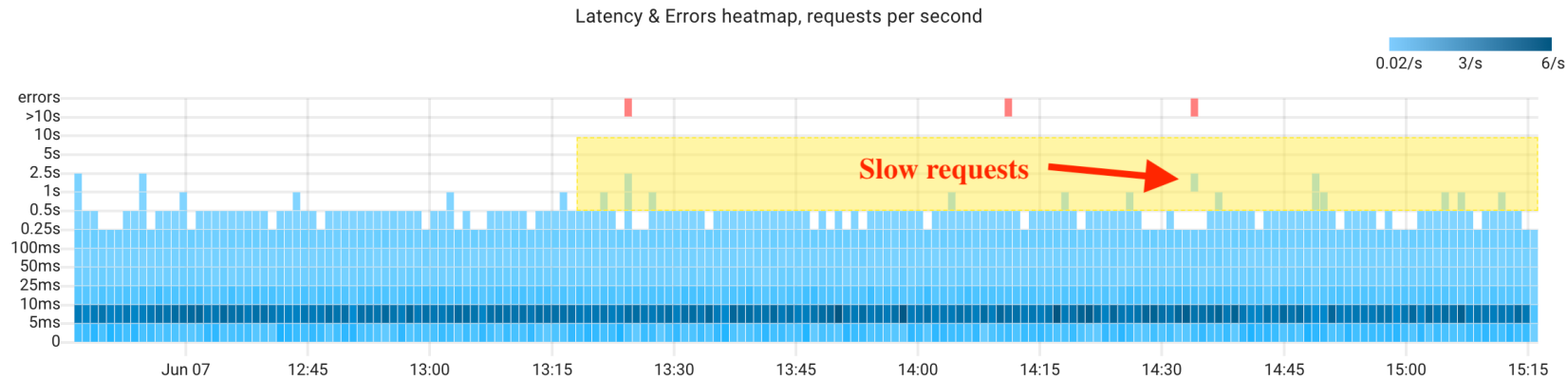
Key Differences:

- **Error Causes:** you don't need to analyze failed request manually one by one
- **Latency Explorer:** identifies reasons of slow requests
- **Attribute comparison:** understand how requests within an anomaly differ from other requests

Distributed tracing: error causes



Distributed tracing: latency explorer



OVERVIEW TRACES ERROR CAUSES **LATENCY EXPLORER** COMPARE ATTRIBUTES

Filters: +

Selection: time Jun 07, 13:18 – Jun 07, 15:16 where (trace duration 500ms – 10s) ×

Baseline: other events within the time window

☒ Exclude auxiliary requests (from monitoring, control plane, etc)

View: **FlameGraph** Diff

ⓘ This section shows the latency FlameGraph for the selected traces. A wider frame indicates greater time consumption by that tracing span.

Search

total (22 s, 100%)

front-end: GET /catalog/ {net.host.name=front-end} (10 s, 48%)

front-end: POST /login {net.host.name=front-end} (8 s, 35%)

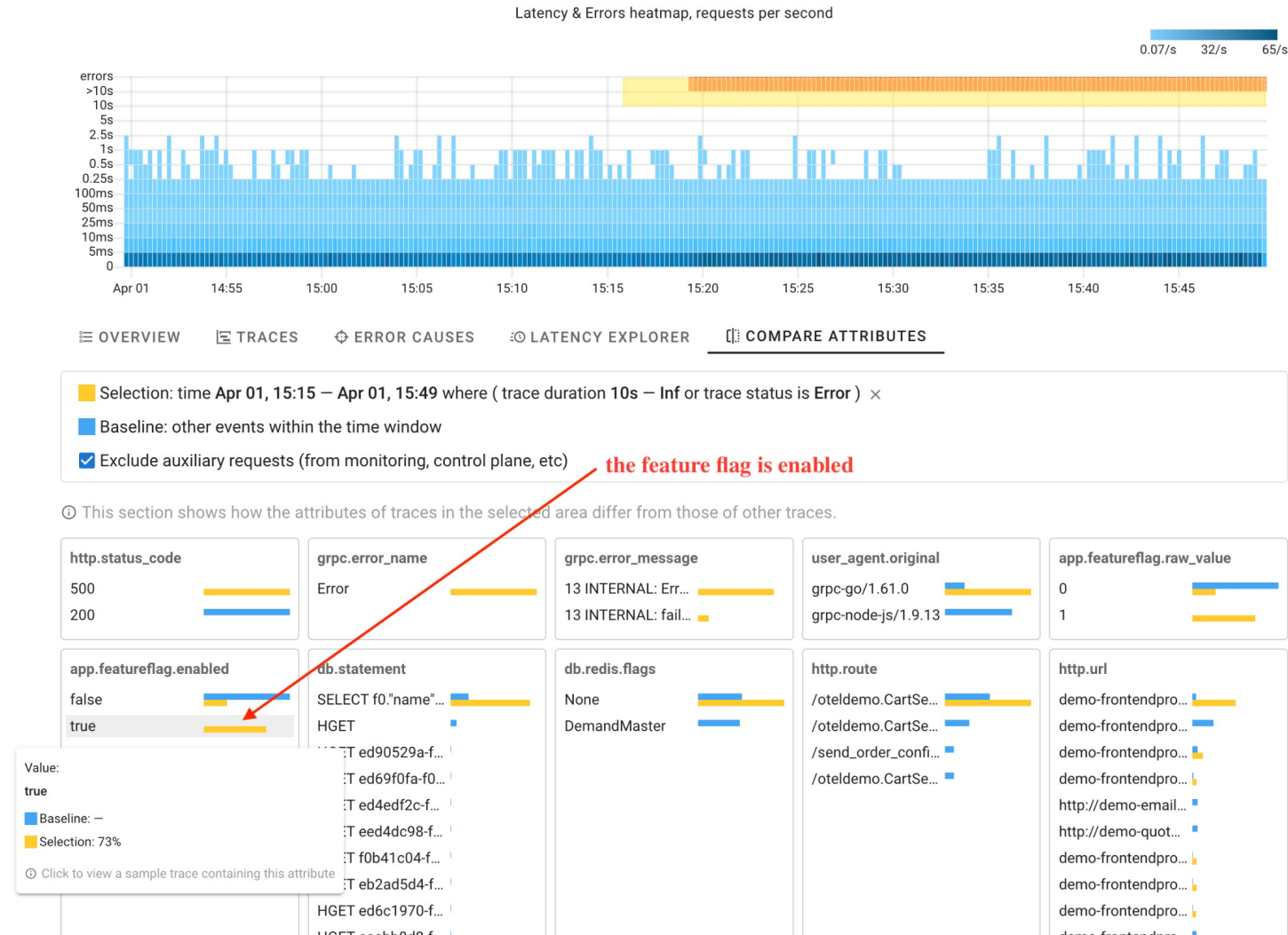
front-end: access_log publish {messaging.operation=publish,messaging.system=kafka} (10 s, 47%)

front-end: HTTP POST {net.peer.name=order} (7 s, 35%)

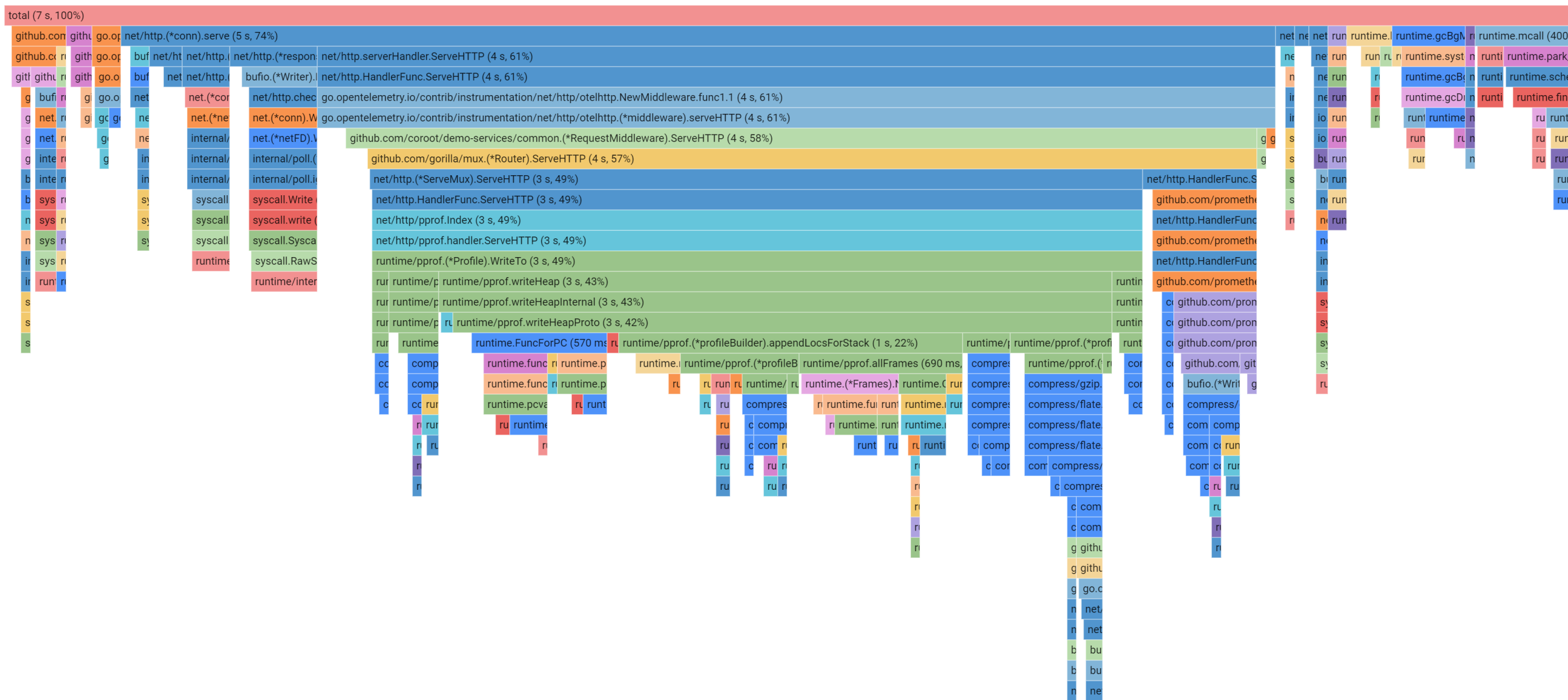
user: POST order: POST /order {net.host.name=order} (10 s, 46%)

order: orders.insert {db.operation=insert,db.system=mongodb,net.peer.name=order-db-0.order-db} (10 s, 46%)

Distributed tracing: attribute comparison



Automagical CPU Profiling with eBPF



Keep Costs in Mind

Overview

- HEALTH
- SERVICE MAP
- TRACES
- NODES
- DEPLOYMENTS
- COSTS

Nodes ⓘ

Node		CPU	Memory	Price	↓ Idle costs
ip-192-168-1-6 m5.large / 2 vCPU / 8GB	usage:			\$73.44/mo on-demand	\$16.19/mo
	request:				
ip-192-168-1-5 m5.large / 2 vCPU / 8GB	usage:			\$73.44/mo on-demand	\$15.15/mo
	request:				
ip-192-168-1-7 m5.large / 2 vCPU / 8GB	usage:			\$73.44/mo on-demand	\$13.91/mo
	request:				
ip-192-168-1-9 m5.large / 2 vCPU / 8GB	usage:			\$73.44/mo on-demand	\$13.70/mo
	request:				
ip-192-168-1-8 m5.large / 2 vCPU / 8GB	usage:			\$73.44/mo on-demand	\$11.42/mo
	request:				
TOTAL				\$367.20/mo	\$70.37/mo

Rows per page: 10 1-5 of 5

Applications ⓘ

Category	↓ Usage costs	Allocation costs	Overprovisioning costs
application	\$75.63/mo	\$62.96/mo	\$22.19/mo
control-plane	\$30.08/mo	\$8.19/mo	\$4.72/mo
monitoring	\$22.56/mo	\$31.66/mo	\$20.12/mo
other	\$0.71/mo	\$8.03/mo	\$7.00/mo
TOTAL	\$128.98/mo	\$110.85/mo	\$54.02/mo

Learn More

<https://coroot.com/>

<https://coroot.com/overview>

<https://github.com/coroot/coroot>

Thank you, Let's connect!

<https://www.linkedin.com/in/peterzaitsev>

<https://twitter.com/PeterZaitsev>

<http://www.peterzaitsev.com>



coroot ➡ :~#